



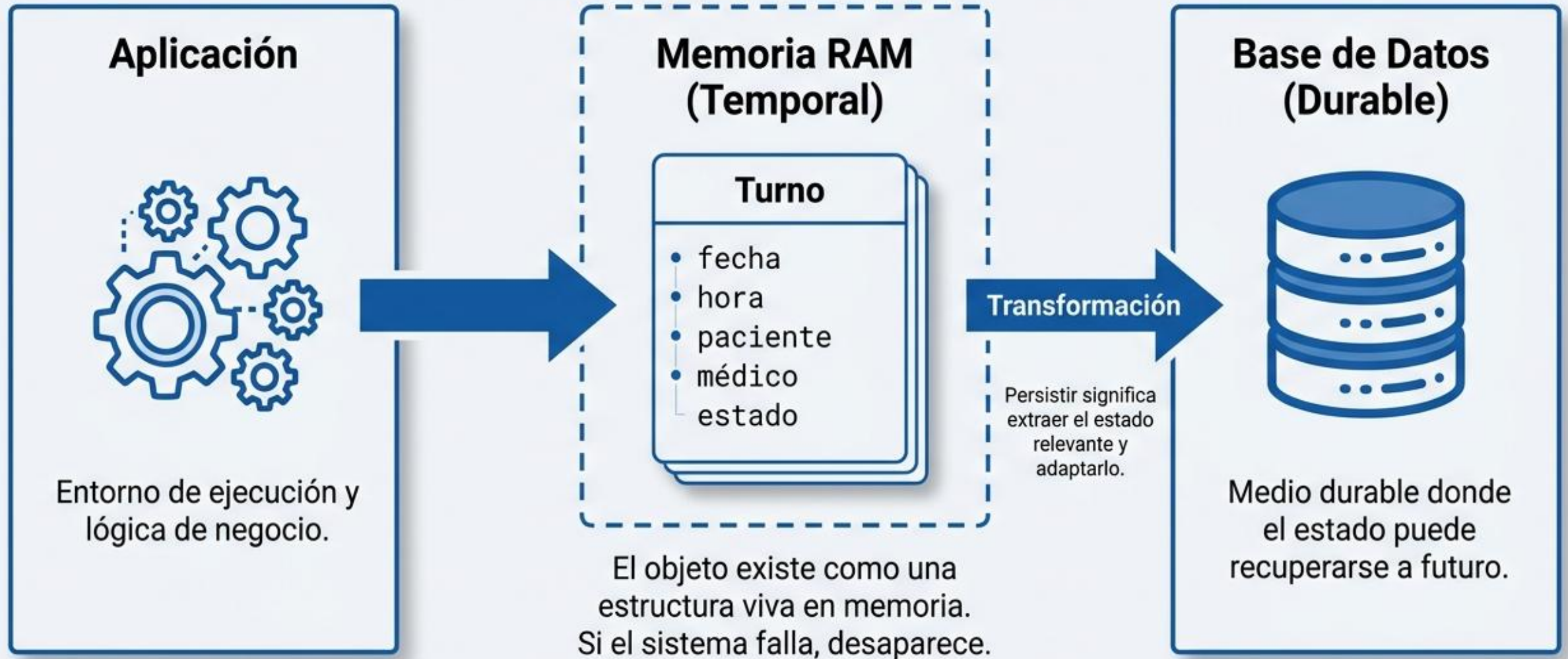
Unidad 4 Clase 9

Replicación y Particionamiento

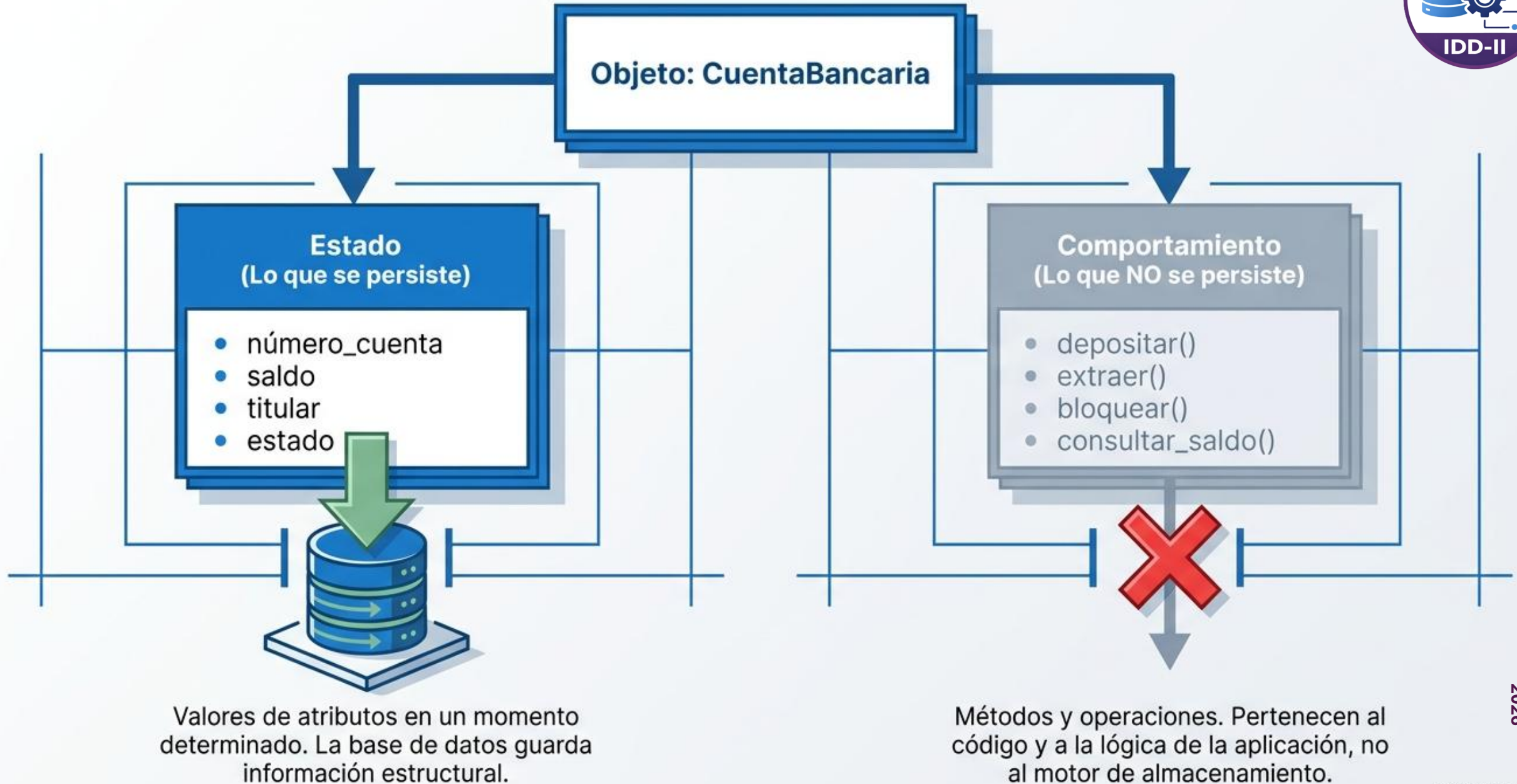
PERSISTENCIA DE OBJETOS INTEGRACION DE APLICACIONES

Comprender los principios de la persistencia de objetos y su integración con bases de datos NoSQL, analizando modelos de replicación y consistencia desde la perspectiva de las aplicaciones.

Del objeto en memoria al dato persistente

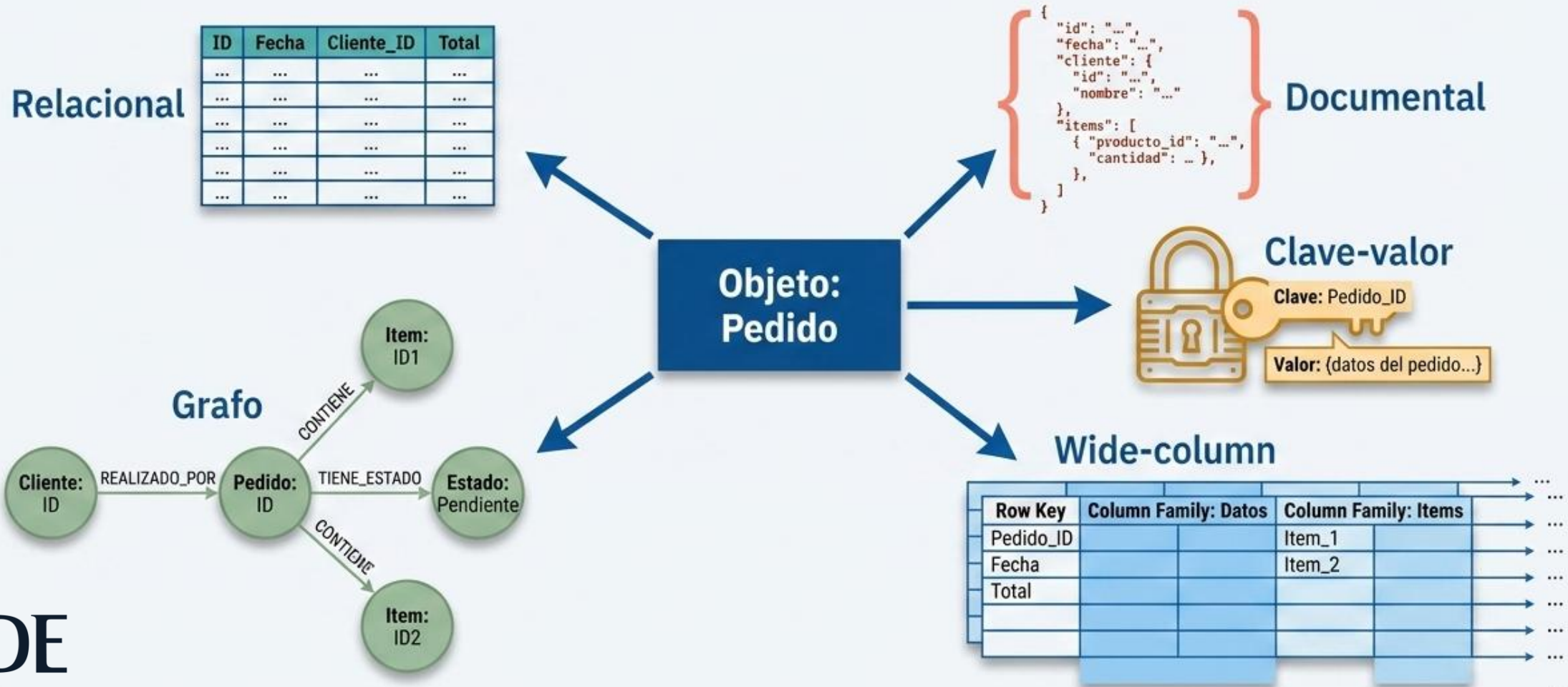


Estado y comportamiento del objeto



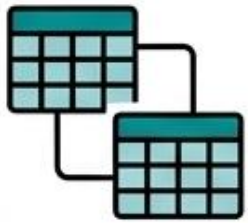
Persistir no es copiar el objeto tal cual

El objeto pertenece a la aplicación; la **base de datos impone su propio modelo de representación** (Transformación).



Representación del objeto en distintos modelos de datos

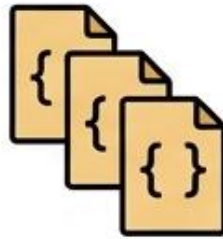
Relacional



Uso Principal:
Tablas, integridad referencial y relaciones rígidamente estructuradas (SQL).

Ejemplo: Entidades financieras y catálogos estáticos.

Documental



Uso Principal:
Objetos jerárquicos o agregados flexibles sin esquema fijo.

Ejemplo: Perfiles de usuario o carritos de compras.

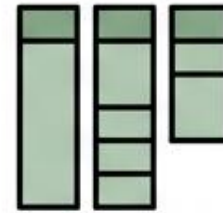
Clave-valor



Uso Principal:
Accesos ultrarrápidos, estados en memoria y cachés.

Ejemplo: Sesiones activas o contadores de visitas.

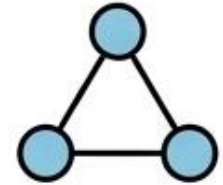
Wide-column



Uso Principal:
Eventos masivos, series temporales y trazabilidad a gran escala.

Ejemplo: Logs de auditoría o telemetría IoT.

Grafo

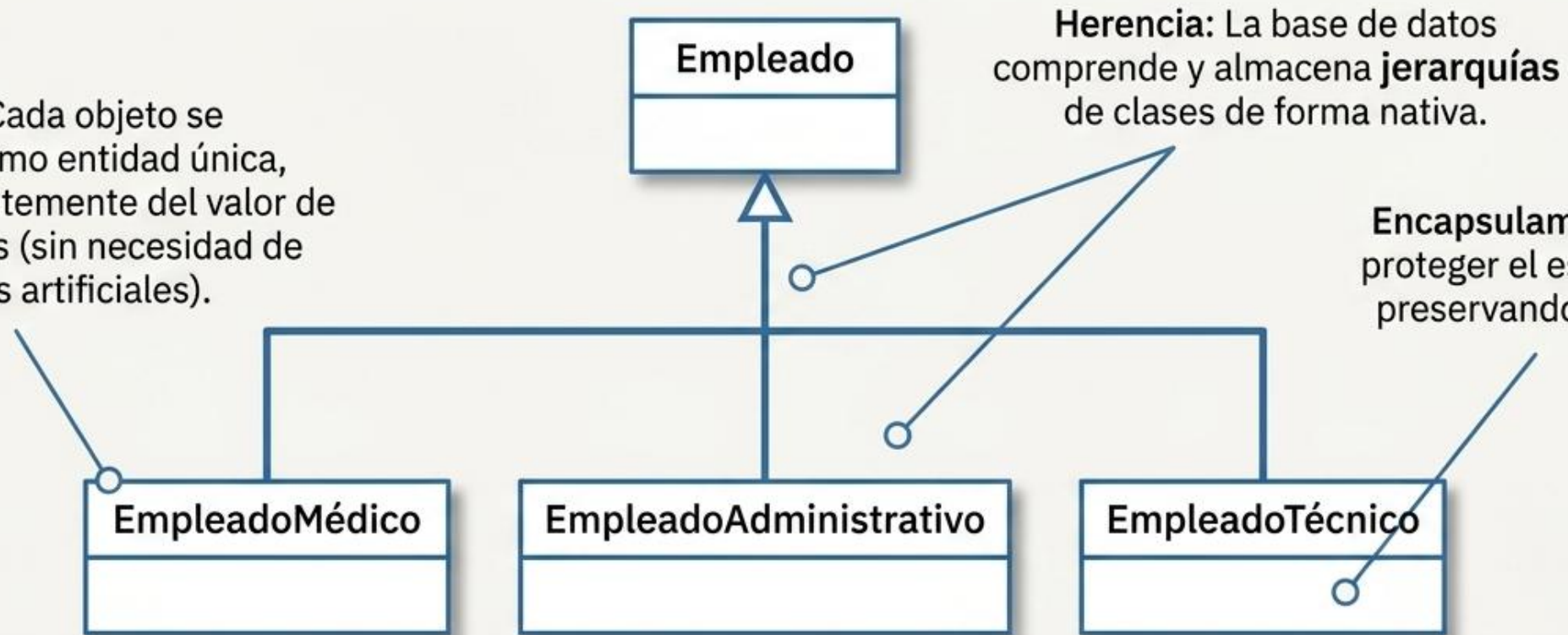


Uso Principal:
Relaciones complejas y patrones de conexión profunda.

Ejemplo: Motores de recomendación o redes sociales.

Bases de datos orientadas a objetos

Identidad: Cada objeto se reconoce como entidad única, independientemente del valor de sus atributos (sin necesidad de Primary Keys artificiales).



Herencia: La base de datos comprende y almacena **jerarquías** de clases de forma nativa.

Encapsulamiento: Busca proteger el estado interno preservando la lógica del paradigma.

Su importancia actual es fundamentalmente conceptual: intentan **reducir a cero la distancia** entre el modelo de programación y el de almacenamiento.

Persistencia de objetos en bases NoSQL

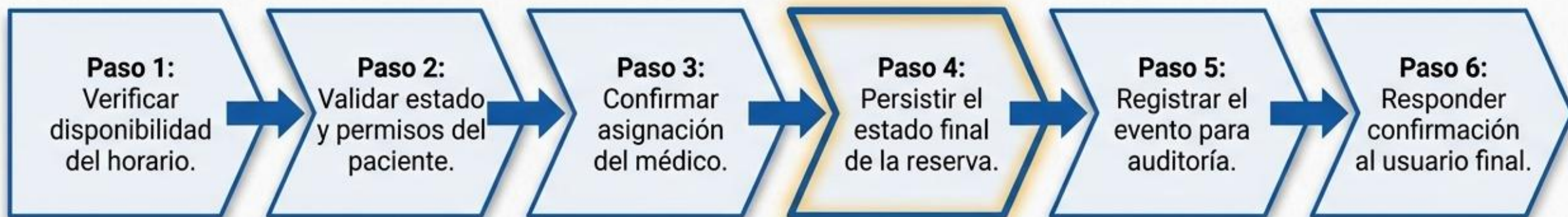
NoSQL (‘No Solamente SQL’) no reemplaza al modelo relacional, sino que optimiza casos donde la estructura tabular es ineficiente.

Necesidad de la Aplicación	Modelo y Tecnología Recomendada	Ejemplo Aplicado
Estructuras ricas, variables y datos anidados.	MongoDB (Documental)	Un pedido complejo que contiene un array de ítems embebidos.
Velocidad extrema en memoria y baja latencia.	Redis (Clave-valor)	Estado de sesión de un usuario o bloqueos temporales.
Volumen masivo de escritura y alta disponibilidad.	Cassandra (Wide-column)	Registro histórico de eventos de una aplicación (logs).
Análisis de patrones y conexiones entre entidades.	Neo4j (Grafo)	Trazar relaciones entre pacientes, médicos y diagnósticos.

Integración entre aplicación y base de datos

Integrar no es solo ejecutar un 'guardar()'. Implica validar reglas, transformar objetos y orquestar pasos lógicos.

Flujo de Reserva de Turno



LA ARQUITECTURA DISTRIBUIDA: La aplicación debe diseñarse asumiendo que habrá fallas de red, demoras de replicación y datos temporalmente desactualizados. Diseñe para la resiliencia.

Mapeo entre objetos y datos persistentes

Del objeto en memoria a la estructura durable

¿Qué es el mapeo?

Transformar un objeto de la aplicación en una estructura persistente y viceversa.

Un mismo objeto adopta formas radicalmente distintas según el modelo de datos (**objeto-relacional**, **objeto-documento**, **objeto-clave**, **objeto-evento**, **objeto-grafo**).



**Objeto Paciente
(En Memoria)**

{nombre, documento,
teléfono, cobertura}



Relacional (SQL)

Se divide en múltiples tablas vinculadas (Ej. Paciente, Turnos, Contacto).



Documental (MongoDB)

Se guarda todo en un único documento JSON consolidado.



Clave-Valor (Redis)

Se guarda temporalmente solo el estado de la sesión.



Grafo (Neo4j)

Es un nodo interconectado con médicos, tratamientos e instituciones.

Decisiones de mapeo y errores frecuentes

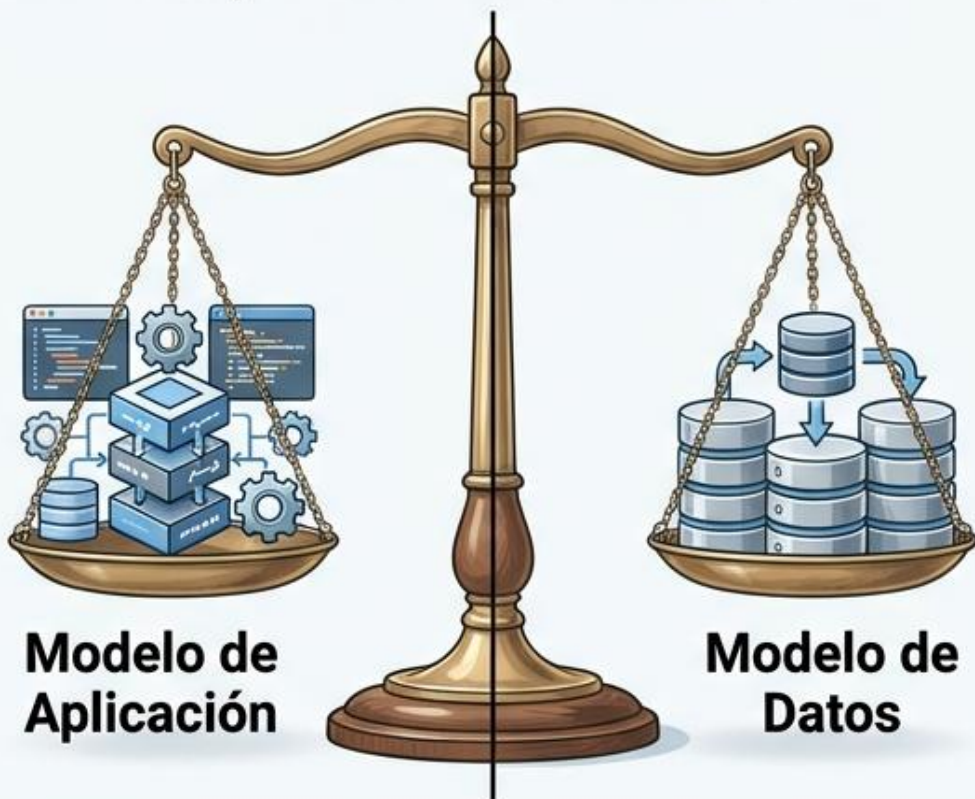
El equilibrio entre el código y el almacenamiento

Mapear exige decisiones arquitectónicas críticas: qué atributos guardar, con qué nombres, qué relaciones mantener y qué nivel de duplicación de datos tolerar.



Error Frecuente 1: Copia Ciega

Guardar el objeto exactamente como está en memoria sin considerar cómo se va a consultar (ej. falta de índices o partición).



**Modelo de
Aplicación**

**Modelo de
Datos**



Error Frecuente 2: Ignorar la Aplicación

Diseñar una base de datos "perfecta" teóricamente, pero que obliga a la aplicación a hacer múltiples peticiones pesadas para armar un objeto.

«El modelo persistente debe ser eficiente para la base de datos y útil para la aplicación»

Replicación de datos

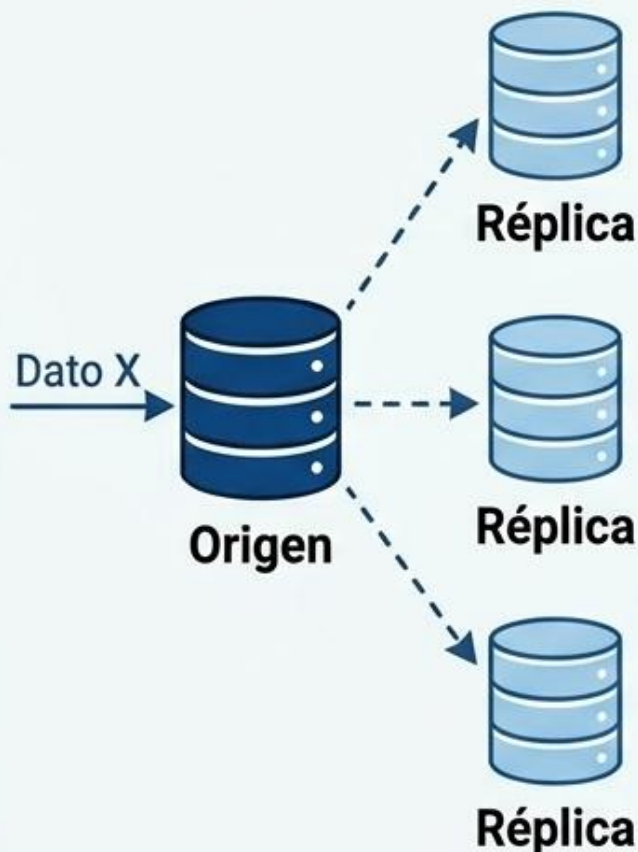
Multiplicando nodos para escalar y sobrevivir

Concepto

Es el proceso de mantener copias idénticas de los datos en más de un nodo. Un nodo es una instancia o servidor dentro de un sistema distribuido.

El Gran Problema:

Mantener coherencia. ¿Cómo garantizamos que todos los nodos tengan exactamente la misma versión del dato en todo momento?



Alta Disponibilidad

Si un nodo falla, el sistema opera con otra copia disponible.



Tolerancia a Fallos

Permite la recuperación y continuidad ante desastres.



Distribución de Carga

Múltiples réplicas asumen simultáneamente el tráfico.



Mejora de Lecturas

Lecturas más rápidas al consultar el nodo geográficamente más cercano.

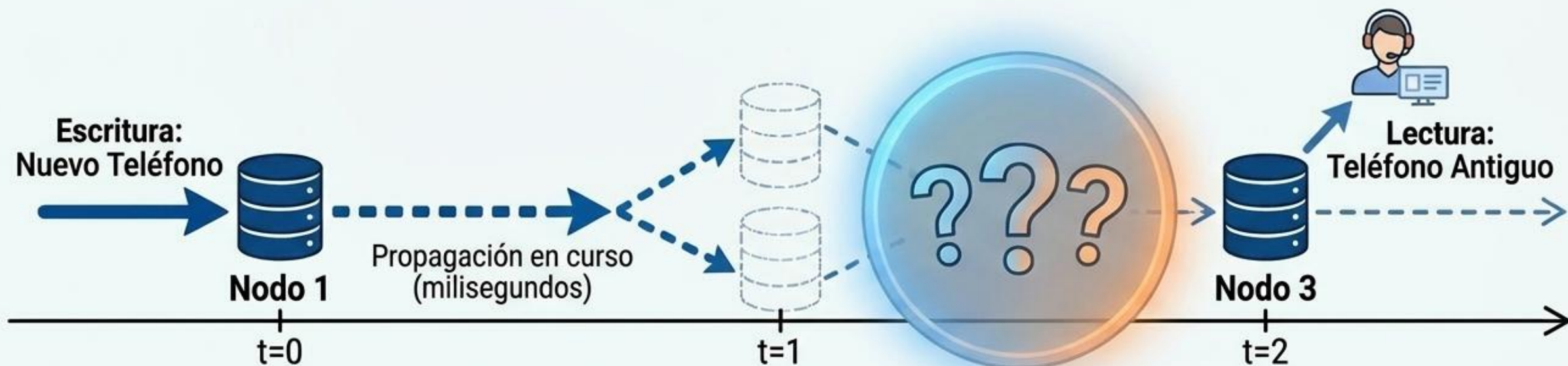
La replicación debe equilibrar consistencia, disponibilidad y partición.

El desafío de replicar datos

Replicar no es solo duplicar, es sincronizar en el tiempo

Si un dato cambia en un nodo, las demás réplicas deben actualizarse. Esta propagación dicta la consistencia del sistema y puede ser:

- Inmediata o diferida
- Sincrónica o asincrónica
- Confirmada por todos los nodos o solo por una mayoría



Ejemplo: Actualización de Teléfono

Un paciente actualiza su teléfono. El nuevo dato ingresa a una de las tres réplicas, pero la propagación toma milisegundos.

¿Cuándo es válido el cambio?

¿Alcanza con que un nodo registre la escritura? ¿Dos? ¿Los tres? Si un usuario lee del Nodo 3 antes de la sincronización, verá el teléfono antiguo (Lectura desactualizada).

«El modelo persistente debe ser eficiente para la base de datos y útil para la aplicación»

Modelo Master-Slave (Primario-Secundario)

Control centralizado de la verdad

El nodo primario recibe de forma exclusiva todas las escrituras. Los nodos secundarios replican los datos desde el primario y se utilizan para lecturas, respaldos o procesamiento de reportes.

Ventajas

- ✓ - Simplicidad arquitectónica.
- ✓ - Sin conflictos de escritura (control centralizado).
- ✓ - Excelente distribución de cargas de lectura analíticas.



Riesgos y Desafíos

- ⚠ - Punto crítico de fallo: si el primario cae, se frena la escritura.
- ⚠ - Requiere mecanismos de 'Failover' complejos.
- ⚠ - Riesgo de lecturas desactualizadas si la replicación es asincrónica.

«El modelo persistente debe ser eficiente para la base de datos y útil para la aplicación»

Modelo Peer to Peer (Entre Pares)

Descentralización y escalabilidad horizontal absoluta

No existe un único nodo principal. Todos los nodos del cluster tienen roles equivalentes y pueden aceptar tanto lecturas como escrituras distribuyendo la información.

Ventajas

- ✓ - Alta disponibilidad: evita la dependencia de un nodo central (SPOF).
- ✓ - Escalabilidad horizontal extrema.
- ✓ - Tolerancia superior ante fallas de hardware.



Desafíos

- ⚠ - Coordinación compleja del cluster.
- ⚠ - Alto riesgo de conflictos de escritura simultáneos.
- ⚠ - Resolución de versiones de datos divergentes.

«El modelo persistente debe ser eficiente para la base de datos y útil para la aplicación»

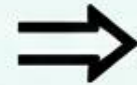
Modelo Quorum

Consenso distribuido: la regla de la mayoría

RF=3

+

Quorum=2



Escritura Válida

El sistema exige una cantidad mínima de confirmaciones (Quorum) para considerar válida una lectura o escritura. No se espera a todos los nodos, pero tampoco basta con uno solo.

Ejemplo Práctico

Si el Factor de Replicación (RF) es 3, un Quorum de 2 asegura que la mayoría posee el dato antes de confirmar el éxito a la aplicación.



Equilibrio (Trade-off)

Baja Latencia / Alta Disponibilidad

(Quorum Menor = Mayor velocidad, riesgo de datos viejos)

Consistencia Fuerte

(Quorum Mayor = Máxima consistencia, mayor lentitud)

«El modelo persistente debe ser eficiente para la base de datos y útil para la aplicación»

Consistencia de datos en sistemas distribuidos

¿Cuando una aplicación modifica un dato, todos los usuarios y servicios ven inmediatamente el mismo valor?



Definimos consistencia como el grado en que los datos se mantienen correctos, actualizados y coherentes entre todas las réplicas del sistema.

Consistencia Eventual: Tolerando la asincronía

Si no hay nuevas modificaciones, todas las réplicas terminarán convergiendo al mismo valor.



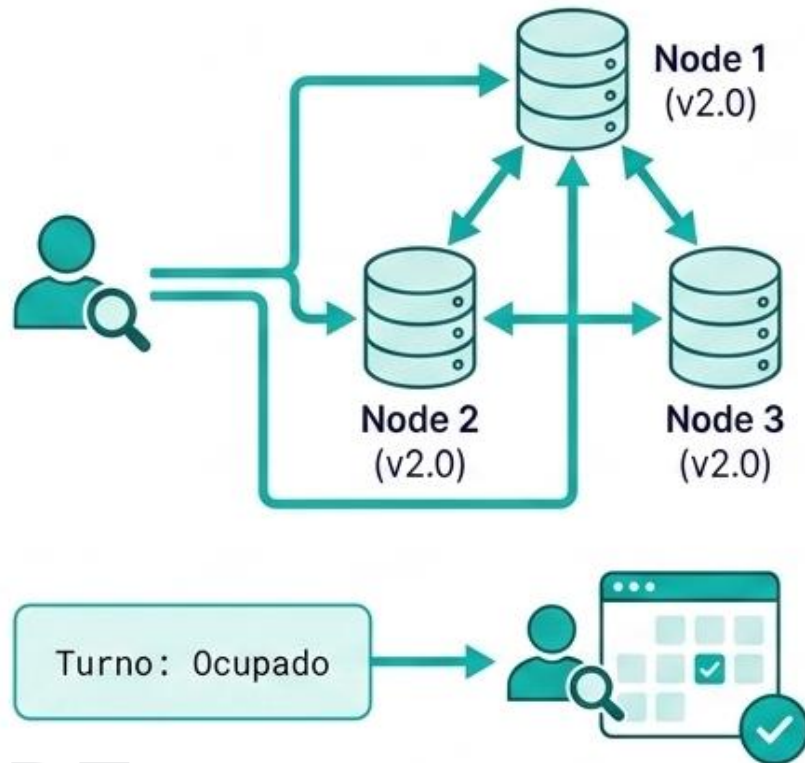
No significa **descontrol**; es una decisión de diseño para **priorizar disponibilidad y baja latencia** en datos no críticos.

Consistencia plena de lectura (Strong Read)

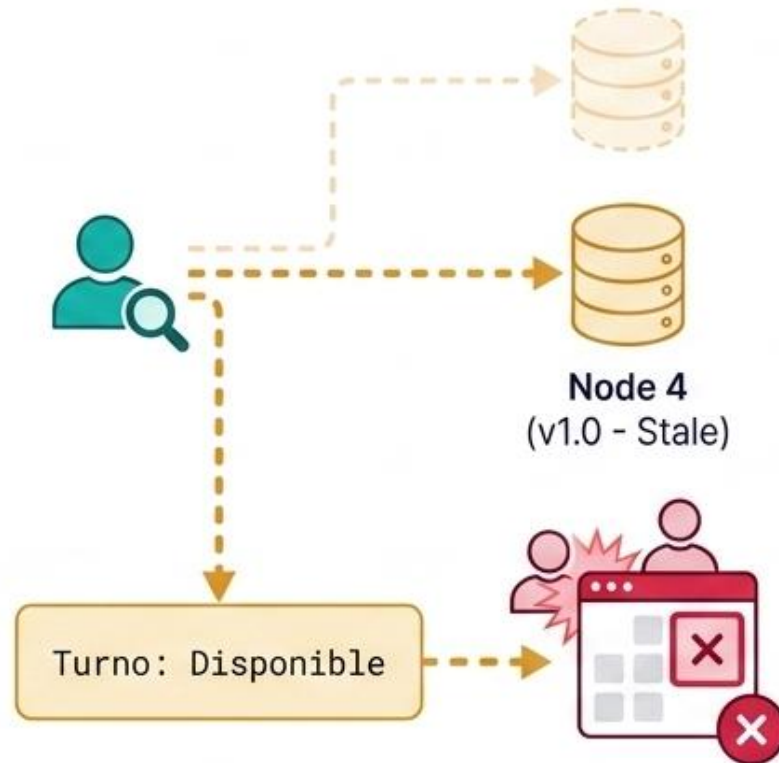
La aplicación exige leer la versión más actualizada y válida luego de una actualización confirmada.



Lectura Segura



Lectura Desactualizada



Casos de uso obligatorios:



Disponibilidad de turnos médicos.



Stock disponible para venta.

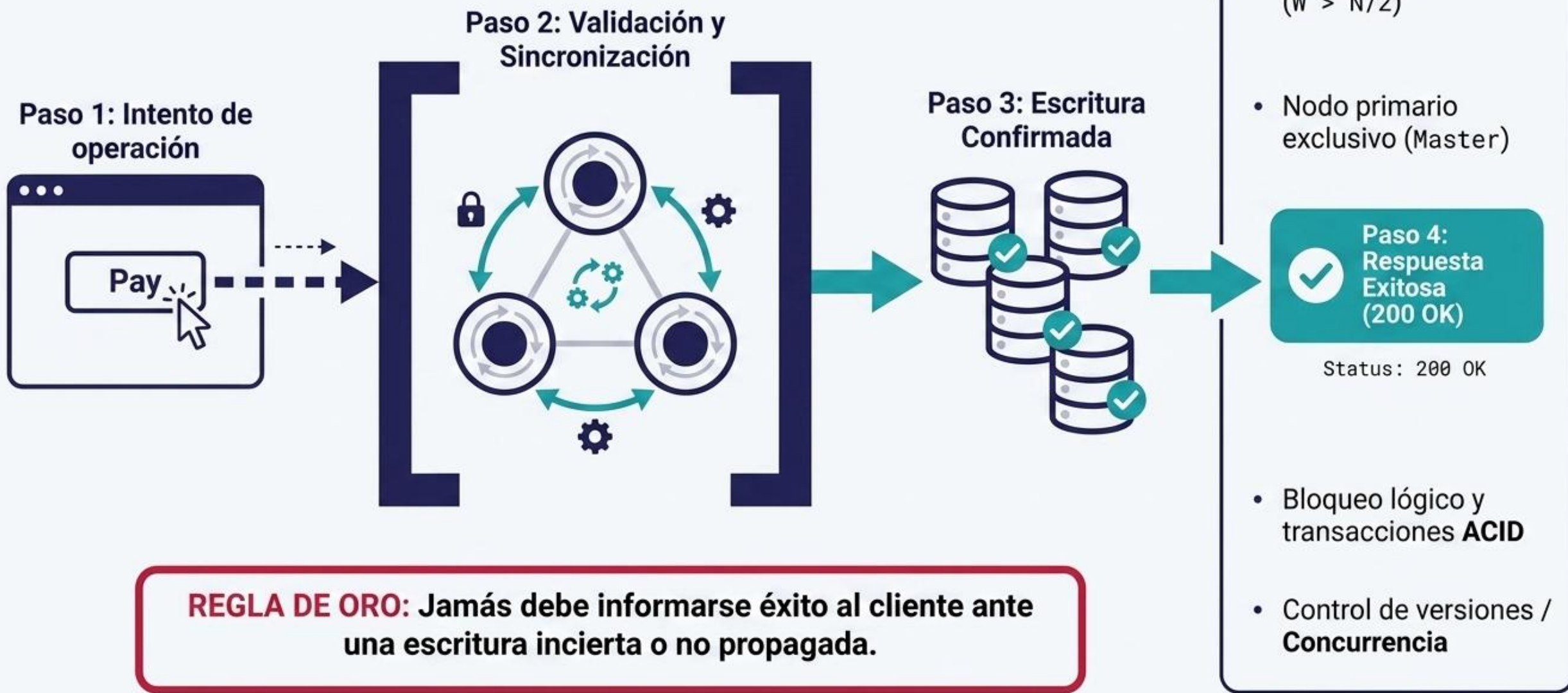


Saldo bancario.

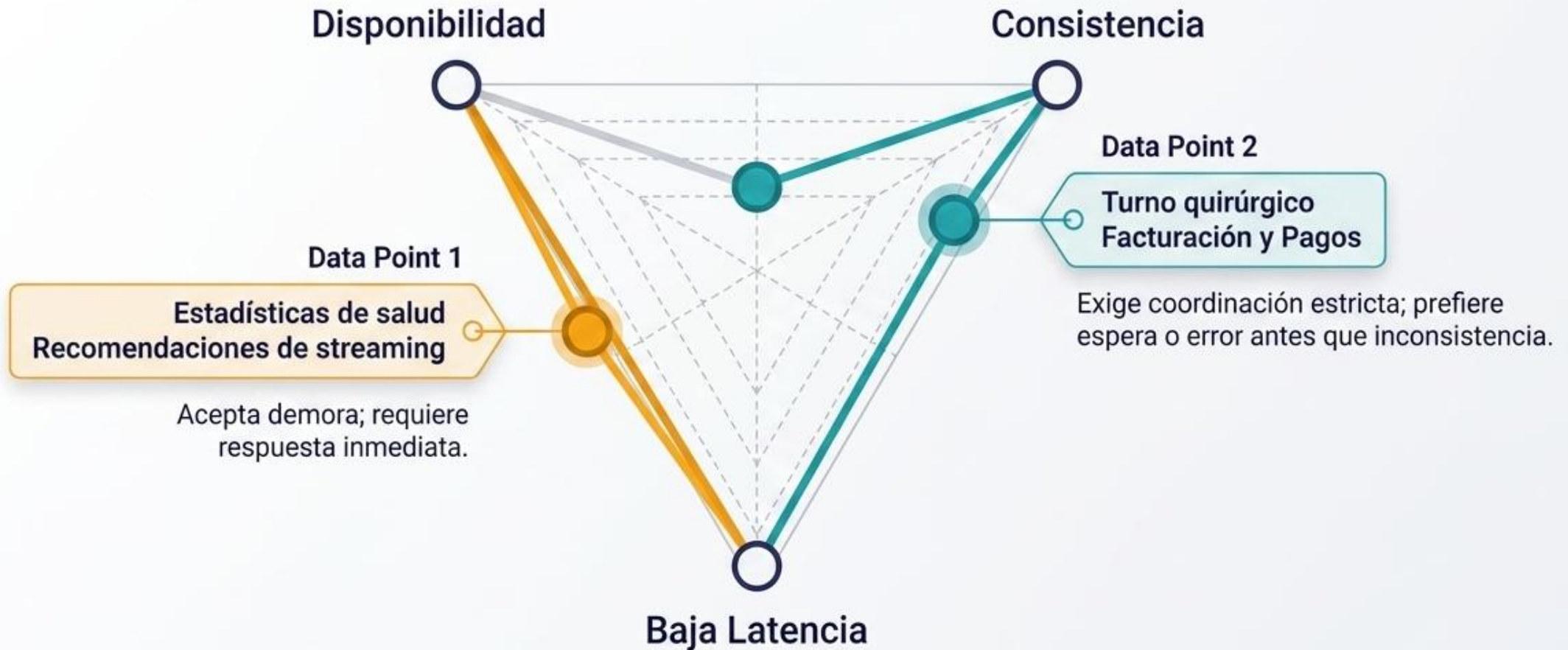


Permisos de seguridad y acceso.

Consistencia plena de escritura (Strong Write)



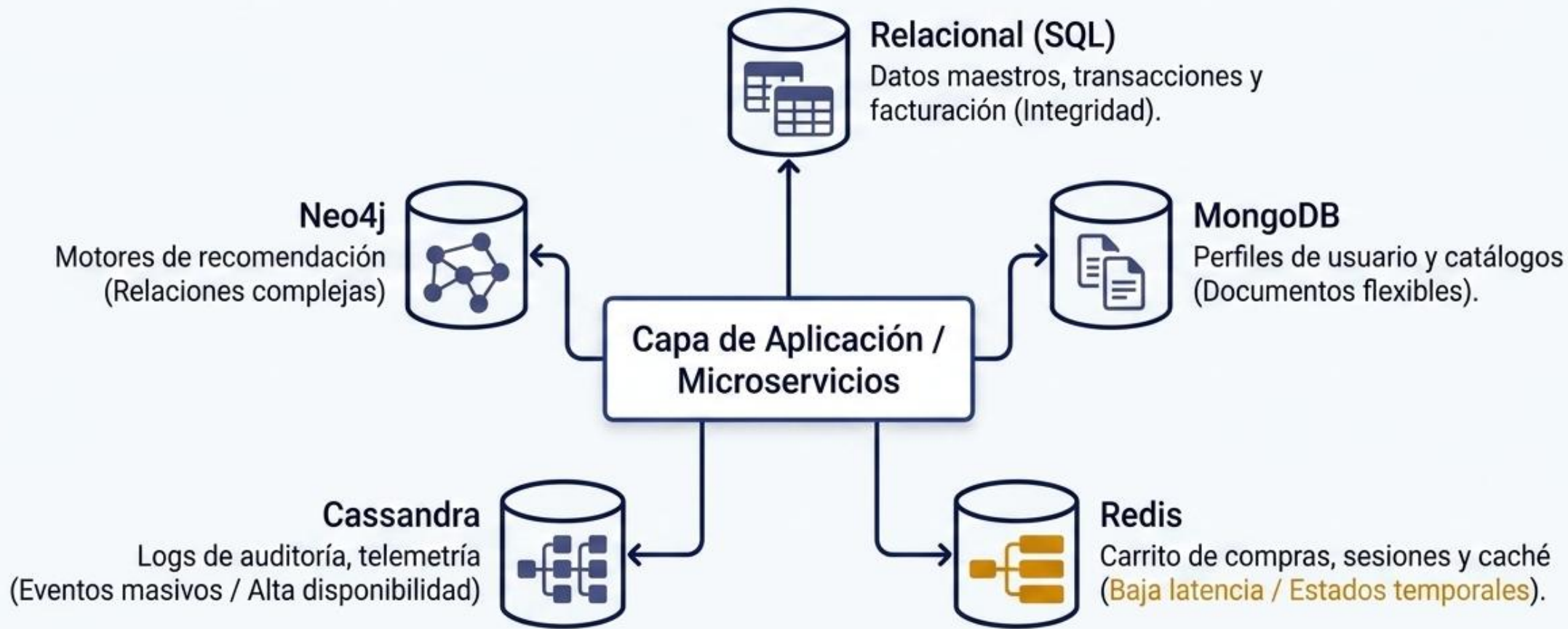
La tensión sistémica: Disponibilidad vs. Consistencia



Replicar mejora la disponibilidad y tolerancia a fallos, pero exige coordinación. Más consistencia requiere más espera (latencia). Más disponibilidad acepta lecturas temporalmente desactualizadas.

Persistencia políglota: Arquitectura de ecosistemas

Asignar cada tipo de dato al motor que mejor resuelve su volumen, latencia y estructura.



No se trata de usar múltiples tecnologías por moda, sino de justificar técnicamente cada modelo para evitar complejidad innecesaria.

Síntesis aplicada:

Diseño orientado al dominio



	Relacional (Consistencia Fuerte)	Documental (Flexibilidad)	Clave-Valor (Baja Latencia / Eventual)	Columnar/Masivo (Trazabilidad)
Caso Clínico (Reservas Médicas)	Gestión de turnos y pagos (Cero colisiones).	Historia clínica electrónica del paciente.	Recordatorios de turnos temporales.	Auditoría inmutable accesos médicos.
Caso Retail (E-Commerce)	Pasarela de pago y stock crítico.	Catálogo de productos y descripciones.	Carrito de compras activo (Redis).	Registro de clics y estadísticas de navegación para recomendaciones.

No existe una persistencia universal. La arquitectura correcta surge de comprender el impacto del dato en el negocio.



Práctica de laboratorio:

Integración MongoDB

+

Redis para gestión de reservas

Una plataforma de reservas médicas permite que los pacientes consulten turnos disponibles y reserven una cita con un profesional. El sistema debe evitar que dos pacientes reserven el mismo turno al mismo tiempo. Además, debe guardar la reserva confirmada de manera persistente y manejar temporalmente el bloqueo de un turno mientras un paciente completa la operación. El problema central es que la aplicación necesita combinar dos necesidades diferentes:

- Por un lado, necesita persistencia durable para registrar pacientes, médicos, turnos y reservas confirmadas.
- Por otro lado, necesita velocidad y control temporal para bloquear un turno durante algunos minutos mientras el usuario confirma la reserva.
- Por este motivo, se propone una solución integrada con MongoDB y Redis.



PROXIMA CLASE



Clase 10 - Acceso a datos NoSQL desde aplicaciones

Tema:

Acceso a estructuras NoSQL desde aplicaciones.
Herramientas de conectividad.
Frameworks de persistencia.
